

CS 133 Lab 3 Report: CUDA Based Parallel CNN

Name: Alexander Neary

UID: 805935345

Date: May 2026

1. Data and Computation Partitioning

1.1 Data Partitioning

The implementation utilizes Shared Memory Tiling to distribute input data across the GPU's streaming multiprocessors.

- **Input Image:** The 228 x 228 input is divided into overlapping tiles of size `TILE_H` x `TILE_W`. These tiles are loaded into `__shared__` memory by all threads in a block acting in parallel.
- **Weights and Bias:** Filter weights and channel biases are accessed from global memory. Each thread block focuses on one of the 256 output channels at a time.
- **Output:** The final 112 x 112 output is partitioned such that each thread is responsible for exactly one pixel in the final pooled result.

1.2 Computation Partitioning

The computation is parallelized using a 3D grid of thread blocks

- **Thread Level:** Each thread calculates a 2 x 2 window of the convolution, applies the ReLU activation, and then performs Max-Pooling to produce a single output value.
- **Block Level:** Threads are organized into 16 x 16 blocks. This configuration allows 256 threads to work together to load a shared memory tile, maximizing memory coalescing.
- **Grid Level:** The grid dimensions (7 x 7 x 256) ensure that every spatial location and every output channel is covered by a thread block.

1.3 Strategy Justification

I chose a fused-kernel strategy with shared memory tiling. This is justified by the NVIDIA T4 GPU's high memory latency compared to its computational speed. By loading data into shared memory once and performing Conv, ReLU, and Max-Pool before writing to global memory, we significantly reduce the bandwidth bottleneck.

2. Optimization Techniques

2.1 Applied Optimizations

- **Kernel Fusion:** Instead of launching separate kernels for convolution, ReLU, and pooling, all operations are performed in a single pass to keep data in fast registers.
 - **Shared Memory Tiling:** Input data is loaded into shared memory to allow multiple threads to reuse the same pixels for overlapping 5 x 5 convolution windows.
 - **Thread-Level Data Reuse:** Each thread calculates four convolution results (2 x 2 area) locally. This increases the computational intensity per thread and reduces the number of times we need to fetch weights.
 - **Memory Coalescing:** The tile-loading logic uses a strided loop, ensuring that 32 threads (a warp) access consecutive memory addresses simultaneously.
-

3. Performance Evaluation

3.1 Best Configuration and GPU Specs

- **Best Parameters:** Blocks per grid: 7 x 7 x 256; Threads per block: 16 x 16 x 1
- **T4 GPU Specifications:** The NVIDIA T4 holds 40 Streaming Multiprocessors (SMs) and 64 CUDA cores per SM.
- **Analysis:** My chosen block size of 64 threads matches the number of cores per SM, leading to high occupancy. The grid size is large enough to ensure all 40 SMs stay fully utilized throughout the execution.

3.2 Performance Incremental Table

The following results track the performance gained as optimizations were added on the **m4dn.xlarge** AWS instance.

Optimization Stage	GFlops	Performance Range	Improvement
Sequential Adaptation	13	E	Initial port of the CPU code to a basic CUDA kernel. Performance is low due to limited thread utilization.
Parallel Global Memory	260	B	Distributed work across a 3D grid. Each thread performs one 5 x 5 convolution using global memory fetches.
Kernel Fusion	640	A	Fused ReLU and Max-Pooling into the same kernel. This kept data in registers and eliminated intermediate global memory writes.

Shared Memory Tiling	1130	A++	Introduced __shared__ float tile[TILE_H][TILE_W] to cache input data. This enabled massive data reuse and achieved peak throughput.
-------------------------	------	-----	---

Performance Range achieved: **A++**

3.3 Grid/Block Configuration Table

Performance results for 3 x 3 configurations:

Block Dim	Grid Dim	GFlops	Performance Range
4 x 4 x 1	28 x 28 x 256	523	A
8 x 8 x 1	14 x 14 x 256	960	A+
16 x 16 x 1	7 x 7 x 256	1130	A++
8 x 4 x 1	14 x 28 x 256	814	A
4 x 8 x 1	28 x 14 x 256	790	A
16 x 8 x 1	7 x 14 x 256	1120	A++
8 x 16 x 1	14 x 7 x 256	983	A+
2 x 2 x 1	56 x 56 x 256	167	D
8 x 2 x 1	14 x 56 x 256	436	B